Lab2_custom_dataflow

February 15, 2026

1 Lab 2: Building a Custom Dataflow Pipeline with Apache Beam

Objective: This notebook provides a complete, end-to-end guide for building and launching a custom data pipeline using the Apache Beam SDK. The pipeline will read a CSV file of historical stock data from Google Cloud Storage (GCS), perform necessary transformations (like reformatting dates and cleaning data), and load the processed data into a new table in Google BigQuery. This lab simulates a real-world data engineering task and teaches the fundamentals of creating robust, serverless ETL (Extract, Transform, Load) jobs on Google Cloud Platform.

2 Cell 1: Install Required Libraries and Resolve Dependencies

Purpose: Before we can build our pipeline, we need to install the necessary Python libraries. We will install apache-beam with the [gcp] extra, which includes all the components needed to run our pipeline on the Google Cloud Dataflow service.

Important Note on Dependencies: Google Colab comes with many pre-installed packages. Sometimes, these can conflict with the specific versions required by new libraries. In this case, apache-beam requires a newer version of a package called dill. To solve this, we will install both apache-beam and the required version of dill in a single command, which helps the package manager resolve the conflict correctly.

Action Required:

Run this cell to install the libraries.

After the installation completes, you must restart the Colab runtime. This ensures that the newly installed package versions are loaded correctly. Go to the menu and select Runtime > Restart runtime.

[1]:
```
# --- Install Apache Beam and its dependencies in a single command ---
# This command installs the Apache Beam SDK with Google Cloud Platform support
# and simultaneously ensures that the 'dill' package is at version 0.3.7 or␣
 ↪newer,
# which resolves a common dependency conflict in the Colab environment.

!pip install --upgrade pip
!pip install --upgrade dill
!pip install --upgrade google-cloud-bigquery
!pip install --quiet apache-beam[gcp]
```

```
!pip install crcmod
```

```
Requirement already satisfied: pip in /usr/local/lib/python3.12/dist-packages
(26.0.1)
Requirement already satisfied: dill in /usr/local/lib/python3.12/dist-packages
(0.4.1)
Requirement already satisfied: google-cloud-bigquery in
/usr/local/lib/python3.12/dist-packages (3.40.1)
Requirement already satisfied: google-api-core<3.0.0,>=2.11.1 in
/usr/local/lib/python3.12/dist-packages (from google-api-
core[grpc]<3.0.0,>=2.11.1->google-cloud-bigquery) (2.29.0)
Requirement already satisfied: google-auth<3.0.0,>=2.14.1 in
/usr/local/lib/python3.12/dist-packages (from google-cloud-bigquery) (2.47.0)
Requirement already satisfied: google-cloud-core<3.0.0,>=2.4.1 in
/usr/local/lib/python3.12/dist-packages (from google-cloud-bigquery) (2.5.0)
Requirement already satisfied: google-resumable-media<3.0.0,>=2.0.0 in
/usr/local/lib/python3.12/dist-packages (from google-cloud-bigquery) (2.8.0)
Requirement already satisfied: packaging>=24.2.0 in
/usr/local/lib/python3.12/dist-packages (from google-cloud-bigquery) (26.0)
Requirement already satisfied: python-dateutil<3.0.0,>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from google-cloud-bigquery)
(2.9.0.post0)
Requirement already satisfied: requests<3.0.0,>=2.21.0 in
/usr/local/lib/python3.12/dist-packages (from google-cloud-bigquery) (2.32.4)
Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.56.2 in
/usr/local/lib/python3.12/dist-packages (from google-api-
core<3.0.0,>=2.11.1->google-api-core[grpc]<3.0.0,>=2.11.1->google-cloud-
bigquery) (1.72.0)
Requirement already satisfied: protobuf!=3.20.0,!=3.20.1,!=4.21.0,!=4.21.1,!=4.2
1.2,!=4.21.3,!=4.21.4,!=4.21.5,<7.0.0,>=3.19.5 in
/usr/local/lib/python3.12/dist-packages (from google-api-
core<3.0.0,>=2.11.1->google-api-core[grpc]<3.0.0,>=2.11.1->google-cloud-
bigquery) (5.29.6)
Requirement already satisfied: proto-plus<2.0.0,>=1.22.3 in
/usr/local/lib/python3.12/dist-packages (from google-api-
core<3.0.0,>=2.11.1->google-api-core[grpc]<3.0.0,>=2.11.1->google-cloud-
bigquery) (1.27.1)
Requirement already satisfied: grpcio<2.0.0,>=1.33.2 in
/usr/local/lib/python3.12/dist-packages (from google-api-
core[grpc]<3.0.0,>=2.11.1->google-cloud-bigquery) (1.65.5)
Requirement already satisfied: grpcio-status<2.0.0,>=1.33.2 in
/usr/local/lib/python3.12/dist-packages (from google-api-
core[grpc]<3.0.0,>=2.11.1->google-cloud-bigquery) (1.65.5)
Requirement already satisfied: pyasn1-modules>=0.2.1 in
/usr/local/lib/python3.12/dist-packages (from google-
auth<3.0.0,>=2.14.1->google-cloud-bigquery) (0.4.2)
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-
packages (from google-auth<3.0.0,>=2.14.1->google-cloud-bigquery) (4.9.1)
```

```
Requirement already satisfied: google-crc32c<2.0.0,>=1.0.0 in
/usr/local/lib/python3.12/dist-packages (from google-resumable-
media<3.0.0,>=2.0.0->google-cloud-bigquery) (1.8.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil<3.0.0,>=2.8.2->google-cloud-bigquery) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.21.0->google-
cloud-bigquery) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-
packages (from requests<3.0.0,>=2.21.0->google-cloud-bigquery) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.21.0->google-
cloud-bigquery) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.21.0->google-
cloud-bigquery) (2026.1.4)
Requirement already satisfied: pyasn1>=0.1.3 in /usr/local/lib/python3.12/dist-
packages (from rsa<5,>=3.1.4->google-auth<3.0.0,>=2.14.1->google-cloud-bigquery)
(0.6.2)
Requirement already satisfied: crcmod in /usr/local/lib/python3.12/dist-packages
(1.7)
```

3 Cell 2: Authenticate and Configure Project Variables

Purpose: This cell handles authentication and sets up all the necessary configuration variables for our project.

Authentication: The auth.authenticate_user() command will prompt you to log in with your Google account. This grants your Colab notebook the necessary permissions to interact with your Google Cloud project's resources (like GCS, BigQuery, and Dataflow).

Configuration: We define all our project-specific names and settings in one place. This makes the code cleaner and easier to adapt for future projects. Each variable is explained in the comments.

[ ]:
```
"Date:TIMESTAMP,Open:FLOAT,High:FLOAT,Low:FLOAT,Close:FLOAT,Volume:
↪INTEGER,Dividends:FLOAT,Stock Splits:FLOAT"
```

[7]:
```
from google.colab import auth
import os

# Authenticate your Google account
auth.authenticate_user()

# --- CONFIGURATION ---
PROJECT_ID = "purdue-ms-ds-bruno-cavagnaro"
REGION = "us-central1"

# Use the buckets and dataset you created previously
```

```
GCS_INPUT_BUCKET = "brunocavagnaro-lab1-bucket"
GCS_FILE_NAME = "NVidia_stock_history.csv"
GCS_TEMP_BUCKET = f"{GCS_INPUT_BUCKET}-temp" # The temp bucket from before
BIGQUERY_DATASET = "nvidia_test_bruno" # The dataset you created before
BIGQUERY_TABLE = "nvidia_prices_from_beam" # A new table name

# Define the full paths for our resources
input_file = f"gs://{GCS_INPUT_BUCKET}/{GCS_FILE_NAME}"
temp_location = f"gs://{GCS_TEMP_BUCKET}/temp"
table_spec = f"{PROJECT_ID}:{BIGQUERY_DATASET}.{BIGQUERY_TABLE}"

# Define the BigQuery table schema as a string
# This matches the header of your CSV file
table_schema = "Date:DATE,Open:FLOAT,High:FLOAT,Low:FLOAT,Close:FLOAT,Volume:
 ↪INTEGER,Dividends:FLOAT,Stock Splits:FLOAT"

print("Configuration is complete.")
```

```
Configuration is complete.
```

[34]:
```
GCS_TEMP_BUCKET
```

[34]:
```
'purdue-ms-ds-bruno-cavagnaro-temp'
```

4 Cell 3: Write the Custom Apache Beam Pipeline Script

Purpose: This is the core of our lab. The %%writefile command is a special Colab "magic" command that writes the entire content of the cell to a Python file named beam_pipeline.py. This script contains the logic for our data pipeline.

Key Concepts in the Script:

parse_csv function: This is our custom transformation logic. For each line read from the CSV, it performs several crucial steps:

Skips the Header: It checks for and ignores the header row.

Parses the Line: It splits the comma-separated line into individual fields.

Cleans and Formats Data: It removes dollar signs from financial data and, most importantly, reformats the date from MM/DD/YYYY to YYYY-MM-DD, which is the required format for BigQuery's DATE data type.

Error Handling: A try-except block gracefully handles any rows that are malformed, preventing a single bad row from crashing the entire pipeline.

Returns a Dictionary: It structures the cleaned data into a Python dictionary, where the keys match our BigQuery table's column names.

run() function: This function defines and executes the pipeline.

PipelineOptions: Configures the pipeline to run on the Dataflow service (DataflowRunner) and specifies our project, job name, region, and temporary storage locations.

beam.Pipeline: This is the main pipeline object. The | (pipe) operator is used to chain together different processing steps.

ReadFromText: Reads the source CSV file from GCS line by line.

FlatMap(parse_csv): Applies our custom parse_csv function to every line.

WriteToBigQuery: Writes the final, processed data to our specified BigQuery table. It also defines the table schema and sets rules for creating the table if it doesn't exist (CREATE_IF_NEEDED) and for overwriting existing data (WRITE_TRUNCATE).

[17]:
```
"Date:DATE,Open:FLOAT,High:FLOAT,Low:FLOAT,Close:FLOAT,Volume:INTEGER,Dividends:
↪FLOAT,Stock Splits:FLOAT"
```

[17]:
```
'Date:DATE,Open:FLOAT,High:FLOAT,Low:FLOAT,Close:FLOAT,Volume:INTEGER,Dividends:
FLOAT,Stock Splits:FLOAT'
```

[18]:
```
%%writefile beam_pipeline.py
import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions
import re
from datetime import datetime

# --- CORRECTED TRANSFORMATION FUNCTION ---
def parse_csv(line):
    line = line.replace("\n", "")
    header = "Date,Open,High,Low,Close,Volume,Dividends,Stock Splits"
    if line.strip() == header:
        return [] # Skip the header row

    fields = re.split(r',(?=(?:[^"]*"[^"]*")*[^"]*$)', line)

    try:
        # --- FIX: Reformat the date string ---
        # Original date format is MM/DD/YYYY
        date_obj = datetime.strptime(fields[0], "%Y-%m-%d %H:%M:%S%z")
        # New format is YYYY-MM-DD, which BigQuery requires for DATE types
        formatted_date = date_obj.strftime('%Y-%m-%d')

        # Create a dictionary for each row
        return [{
            "Date": formatted_date, # Use the correctly formatted date
            "Open": float(fields[1].replace('$', '')),
            "High": float(fields[2]),
            "Low": float(fields[3].replace('$', '')),
            "Close": float(fields[4].replace('$', '')),
            "Volume": int(fields[5].replace('$', '')),
```

```
            "Dividends": float(fields[6].replace('$', '')),
            "Stock Splits": float(fields[7].replace('$', ''))
        }]
    except (ValueError, IndexError) as e:
        # This will catch and log any rows that don't match our format
        print(f"Skipping malformed row: {line} | Error: {e}")
        return []

def run():
    # --- CORRECTED PIPELINE OPTIONS ---
    # We explicitly add the project ID to the options
    options = PipelineOptions(
        runner='DataflowRunner',
        project="purdue-ms-ds-bruno-cavagnaro", # --- FIX: Explicitly set the␣
 ↪project
        job_name='gcs-to-bq-beam-pipeline-v3', # New job name
        staging_location=f'gs://brunocavagnaro-lab1-bucket-temp/staging',
        temp_location=f'gs://brunocavagnaro-lab1-bucket-temp/temp',
        region='us-central1'
    )

    # Define the pipeline
    with beam.Pipeline(options=options) as p:
        (
            p
            | 'ReadFromGCS' >> beam.io.ReadFromText("gs://
 ↪brunocavagnaro-lab1-bucket/NVidia_stock_history.csv")
            | 'ParseAndFormat' >> beam.FlatMap(parse_csv) # Using the corrected␣
 ↪function
            | 'WriteToBigQuery' >> beam.io.WriteToBigQuery(
                table="purdue-ms-ds-bruno-cavagnaro:nvidia_test_bruno.
 ↪nvidia_prices_from_beam",
                schema="Date:DATE,Open:FLOAT,High:FLOAT,Low:FLOAT,Close:
 ↪FLOAT,Volume:INTEGER,Dividends:FLOAT,Stock Splits:FLOAT",
                create_disposition=beam.io.BigQueryDisposition.CREATE_IF_NEEDED,
                write_disposition=beam.io.BigQueryDisposition.WRITE_TRUNCATE
            )
        )

if __name__ == '__main__':
    run()
```

```
Overwriting beam_pipeline.py
```

5 Cell 4: Launch the Dataflow Job

Purpose: This cell executes the beam_pipeline.py script that you just created. The ! symbol tells Colab to run the command in the underlying Linux shell. This command will package up your pipeline code and submit it to the Google Cloud Dataflow service, which will then provision workers and run the job.

[19]:
```
print("Launching the Dataflow job... This may take a minute.")

# Run the beam_pipeline.py script
!python beam_pipeline.py

print("\n---")
print("Job has been submitted to Dataflow.")
print("You can monitor its progress in the GCP console:")
print(f"https://console.cloud.google.com/dataflow/jobs?project={PROJECT_ID}")
```

```
Launching the Dataflow job… This may take a minute.

---
Job has been submitted to Dataflow.
You can monitor its progress in the GCP console:
https://console.cloud.google.com/dataflow/jobs?project=purdue-ms-ds-bruno-
cavagnaro
```

6 Cell 5: Verify the Final Result in BigQuery

Purpose: After waiting a few minutes for the Dataflow job to show "Succeeded" in the GCP console, you can run this final cell. It connects to BigQuery, queries the new table created by your pipeline, and displays the results in a clean pandas DataFrame, confirming that your entire ETL process was successful.

[23]:
```
from google.cloud import bigquery
import pandas as pd

# --- Verify the Final Result in BigQuery ---

# Use the BigQuery client we configured earlier
client = bigquery.Client(project=PROJECT_ID)

# The full ID of the table your Beam pipeline created
dataset = f"{PROJECT_ID}.{BIGQUERY_DATASET}"
table_id = f"{PROJECT_ID}.{BIGQUERY_DATASET}.{BIGQUERY_TABLE}"

# SQL query to select the first 15 rows from your table
sql_query = f"""
SELECT
    *
```

```
FROM
    `{table_id}`
ORDER BY
    Date DESC
LIMIT 15
"""

try:
    # Execute the query and load the results into a pandas DataFrame
    df = client.query(sql_query).to_dataframe()

    print(f"--- Successfully queried data from '{table_id}' ---")
    # Display the DataFrame. Colab will format it nicely.
    display(df)

except Exception as e:
    print(f"An error occurred while querying the table: {e}")
```

```
--- Successfully queried data from 'purdue-ms-ds-bruno-
cavagnaro.nvidia_test_bruno.nvidia_prices_from_beam' ---
```

	Date	Open	High	Low	Close	Volume	\
0	2024-08-28	128.119995	128.330002	122.639999	125.175003	241795982	
1	2024-08-27	125.050003	129.199997	123.879997	128.300003	301726100	
2	2024-08-26	129.570007	131.259995	124.370003	126.459999	331964700	
3	2024-08-23	125.860001	129.600006	125.220001	129.369995	323230300	
4	2024-08-22	130.020004	130.750000	123.099998	123.739998	376189100	
5	2024-08-21	127.320000	129.350006	126.660004	128.500000	257883600	
6	2024-08-20	128.399994	129.880005	125.889999	127.250000	300087400	
7	2024-08-19	124.279999	130.000000	123.419998	130.000000	318333600	
8	2024-08-16	121.940002	125.000000	121.180000	124.580002	302589900	
9	2024-08-15	118.760002	123.239998	117.470001	122.860001	318086700	
10	2024-08-14	118.529999	118.599998	114.070000	118.080002	339246400	
11	2024-08-13	112.440002	116.230003	111.580002	116.139999	312646700	
12	2024-08-12	106.320000	111.070000	106.260002	109.019997	325559900	
13	2024-08-09	105.639999	106.599998	103.430000	104.750000	290844200	
14	2024-08-08	102.000000	105.500000	97.519997	104.970001	391910000	

	Dividends	Stock Splits
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
5	0.0	0.0
6	0.0	0.0
7	0.0	0.0
8	0.0	0.0

```
9          0.0             0.0
10         0.0             0.0
11         0.0             0.0
12         0.0             0.0
13         0.0             0.0
14         0.0             0.0
```

7 Linear Regression (Predicting a Stock's High Price)

Objective: Use Linear Regression to predict a continuous value. In this case, we will predict the High price of a stock for a given day based on its Open, Low, and Close_Last prices.

Action: Run the cells below to create, evaluate, and use the model.

[24]:
```
# --- 1. Create the Linear Regression Model ---

# This SQL statement creates a model named 'nvidia_high_predictor'.
# It uses the LINEAR_REG model type to predict the 'High' column (our label).
# The FEATURES clause specifies which columns to use for the prediction.
create_linear_model_query = f"""
CREATE OR REPLACE MODEL `{dataset}.nvidia_high_predictor`
OPTIONS(model_type='LINEAR_REG', input_label_cols=['High']) AS
SELECT
  Open,
  High,
  Low,
  Close
FROM
  `{table_id}`
"""

# Execute the query to create and train the model.
# This may take a few minutes.
linear_job = client.query(create_linear_model_query)
linear_job.result() # Wait for the job to complete

print(" Linear Regression model created successfully.")

# --- 2. Evaluate the Model ---

# This query uses ML.EVALUATE to see how well our model performed.
# For linear regression, look at metrics like 'r2_score' and␣
 ↪'mean_absolute_error'.
evaluate_linear_model_query = f"""
SELECT
  *
FROM
  ML.EVALUATE(MODEL `{dataset}.nvidia_high_predictor`)
```

```
"""
linear_eval_df = client.query(evaluate_linear_model_query).to_dataframe()
print("\n--- Model Evaluation ---")
display(linear_eval_df)


# --- 3. Make Predictions with the Model ---

# This query uses ML.PREDICT to predict the 'High' price for a few sample days.
# The model will output a 'predicted_High' column.
predict_linear_query = f"""
SELECT
  High AS actual_high,
  predicted_High,
  Open,
  Low,
  Close
FROM
  ML.PREDICT(MODEL `{dataset}.nvidia_high_predictor`,
    (
    SELECT
      Open, High, Low, Close
    FROM
      `{table_id}`
    WHERE
      EXTRACT(YEAR FROM Date) = 2024
    LIMIT 5
    )
  )
"""
linear_predict_df = client.query(predict_linear_query).to_dataframe()
print("\n--- Sample Predictions ---")
display(linear_predict_df)
```

```
 Linear Regression model created successfully.

--- Model Evaluation ---
   mean_absolute_error  mean_squared_error  mean_squared_log_error  \
0              0.04657            0.028438                 0.00003

   median_absolute_error  r2_score  explained_variance
0               0.003409  0.999904            0.999904

--- Sample Predictions ---
   actual_high  predicted_High       Open        Low      Close
0    49.288639       49.743487  49.237647  47.588861  48.161785
```

```
1    48.177781       48.080417  47.478874  47.313894  47.562862
2    48.493744       48.503308  47.760837  47.501871  47.991810
3    49.540607       49.426190  48.455748  48.299766  49.090664
4    52.268254       51.820330  49.505610  49.472614  52.246254
```

8 Logistic Regression (Predicting a High Volume Day)

Objective: Use Logistic Regression for binary classification. We will predict whether a given day was a "high volume" trading day. To do this, we first need to create our target label: is_high_volume. We'll define "high volume" as any day where the volume was greater than the average volume for the entire dataset.

Action: Run the cells below to create a classification model.

[25]:
```sql
# --- 1. Pre-calculate the Average Volume ---
# This step remains the same. We need the average to create our label.
avg_volume_query = f"""
SELECT
    AVG(Volume) as avg_vol
FROM
    `{table_id}`
"""
avg_volume = client.query(avg_volume_query).to_dataframe().iloc[0,0]
print(f"Calculated Average Volume: {avg_volume:,.0f}")


# --- 2. Create a Training View with the Label ---
# This is the new, crucial step. We create a VIEW that contains our final,␣
 ↪clean data.
# This separates our data preparation logic from our model training logic.
create_view_query = f"""
CREATE OR REPLACE VIEW `{dataset}.nvidia_training_data` AS
SELECT
  Open,
  High,
  Low,
  Close,
  -- This is where we create the label that our model will predict.
  CASE
    WHEN Volume > {avg_volume} THEN TRUE
    ELSE FALSE
  END AS is_high_volume
FROM
  `{table_id}`
"""
view_job = client.query(create_view_query)
view_job.result() # Wait for the view to be created
print("\n Training view `nvidia_training_data` created successfully.")
```

```
# --- 3. Create the Model from the Clean View ---
# This CREATE MODEL statement is now much simpler. It has NO TRANSFORM clause.
# It simply selects the features and the label from our new view.
create_logistic_model_query = f"""
CREATE OR REPLACE MODEL `{dataset}.nvidia_volume_classifier`
OPTIONS(model_type='LOGISTIC_REG', input_label_cols=['is_high_volume']) AS
SELECT
  *
FROM
  `{dataset}.nvidia_training_data`
"""
logistic_job = client.query(create_logistic_model_query)
logistic_job.result() # Wait for the job to complete
print("\n Logistic Regression model created successfully.")


# --- 4. Evaluate and Predict (This part remains the same) ---
evaluate_logistic_model_query = f"""
SELECT
  *
FROM
  ML.EVALUATE(MODEL `{dataset}.nvidia_volume_classifier`)
"""
logistic_eval_df = client.query(evaluate_logistic_model_query).to_dataframe()
print("\n--- Model Evaluation ---")
display(logistic_eval_df)


predict_logistic_query = f"""
SELECT
  predicted_is_high_volume,
  predicted_is_high_volume_probs AS probability, -- FIX: Corrected the column␣
 ↪name for probabilities
  * EXCEPT(predicted_is_high_volume, predicted_is_high_volume_probs) -- Exclude␣
 ↪the original prediction columns
FROM
  ML.PREDICT(MODEL `{dataset}.nvidia_volume_classifier`,
    (
    -- We still use the original table for prediction, as the model knows how␣
 ↪to transform it.
    SELECT
      Open, High, Low, Close, Volume
    FROM
      `{table_id}`
    ORDER BY Date DESC
```

```
    LIMIT 5
    )
  )
"""
logistic_predict_df = client.query(predict_logistic_query).to_dataframe()
print("\n--- Sample Predictions ---")
display(logistic_predict_df)
```

```
Calculated Average Volume: 605,280,689

 Training view `nvidia_training_data` created successfully.

 Logistic Regression model created successfully.

--- Model Evaluation ---
```

	precision	recall	accuracy	f1_score	log_loss	roc_auc
0	0.0	0.0	0.61938	0.0	0.655788	0.621378

```
--- Sample Predictions ---
```

	predicted_is_high_volume	\
0	False	
1	False	
2	False	
3	False	
4	False	

	probability	Open	High	\
0	[{'label': True, 'prob': 0.019191596671828258}…	128.119995	128.330002	
1	[{'label': True, 'prob': 0.018897703201250896}…	125.050003	129.199997	
2	[{'label': True, 'prob': 0.018223101068362252}…	129.570007	131.259995	
3	[{'label': True, 'prob': 0.018415149691748112}…	125.860001	129.600006	
4	[{'label': True, 'prob': 0.018756528560751853}…	130.020004	130.750000	

	Low	Close	Volume
0	122.639999	125.175003	241795982
1	123.879997	128.300003	301726100
2	124.370003	126.459999	331964700
3	125.220001	129.369995	323230300
4	123.099998	123.739998	376189100

9 K-Means Clustering (Grouping Similar Trading Days)

Objective: Use K-Means, an unsupervised learning algorithm, to group similar data points into clusters. We will group trading days into 4 clusters based on their price and volume characteristics. Since Volume is on a much larger scale than the price columns, we must standardize our features using ML.STANDARD_SCALER to ensure all features are weighted equally.

Action: Run the cells below to create the clustering model.

[26]:
```
# --- 1. Create the K-Means Clustering Model ---

# This query creates a K-Means model with 4 clusters.
# The TRANSFORM clause is crucial here. It uses ML.STANDARD_SCALER to
# normalize all of our feature columns. This prevents features with large
# scales (like Volume) from dominating the distance calculations.
create_kmeans_model_query = f"""
CREATE OR REPLACE MODEL `{dataset}.nvidia_day_clusterer`
TRANSFORM(
    -- Standardize all features to have a mean of 0 and standard deviation of 1
    ML.STANDARD_SCALER(Open) OVER() AS std_Open,
    ML.STANDARD_SCALER(High) OVER() AS std_High,
    ML.STANDARD_SCALER(Low) OVER() AS std_Low,
    ML.STANDARD_SCALER(Close) OVER() AS std_Close,
    ML.STANDARD_SCALER(Volume) OVER() AS std_Volume
)
OPTIONS(model_type='KMEANS', num_clusters=4) AS
SELECT
  Open,
  High,
  Low,
  Close,
  Volume
FROM
  `{dataset}.nvidia_prices_from_beam`
"""
kmeans_job = client.query(create_kmeans_model_query)
kmeans_job.result()
print("  K-Means Clustering model created successfully.")


# --- 2. See which Cluster Each Day Belongs To ---

# ML.PREDICT for a K-Means model will add a 'CENTROID_ID' column,
# which indicates the cluster number (from 1 to 4) each row belongs to.
predict_kmeans_query = f"""
SELECT
  CENTROID_ID,
  *
FROM
  ML.PREDICT(MODEL `{dataset}.nvidia_day_clusterer`,
    (
    SELECT
      Open, High, Low, Close, Volume
    FROM
```

```
      `{table_id}`
    ORDER BY Date DESC
    LIMIT 10
    )
  )
"""
kmeans_predict_df = client.query(predict_kmeans_query).to_dataframe()
print("\n--- Sample Cluster Assignments ---")
display(kmeans_predict_df)


# --- 3. Analyze the Cluster Centroids ---

# This is the most important step for understanding a K-Means model.
# ML.CENTROIDS shows us the average values for each feature within each cluster.
# By looking at these centroids, we can give each cluster a business meaning.
# For example, one cluster might be "low price, low volume days" while another
# might be "high price, high volatility, high volume days".
analyze_centroids_query = f"""
SELECT
  *
FROM
  ML.CENTROIDS(MODEL `{dataset}.nvidia_day_clusterer`)
"""
centroids_df = client.query(analyze_centroids_query).to_dataframe()
print("\n--- Cluster Centroid Analysis ---")
display(centroids_df)
```

```
 K-Means Clustering model created successfully.

--- Sample Cluster Assignments ---
   CENTROID_ID  CENTROID_ID_1  \
0            1              1
1            1              1
2            1              1
3            1              1
4            1              1
5            1              1
6            1              1
7            1              1
8            1              1
9            1              1

                         NEAREST_CENTROIDS_DISTANCE        Open        High  \
0  [{'CENTROID_ID': 1, 'DISTANCE': 6.444088917610…  130.020004  130.750000
1  [{'CENTROID_ID': 1, 'DISTANCE': 6.414195690288…  125.050003  129.199997
2  [{'CENTROID_ID': 1, 'DISTANCE': 5.717092115024…  118.760002  123.239998
```

3	[{'CENTROID_ID': 1, 'DISTANCE': 6.018271661112…	121.940002	125.000000
4	[{'CENTROID_ID': 1, 'DISTANCE': 6.516998946038…	125.860001	129.600006
5	[{'CENTROID_ID': 1, 'DISTANCE': 6.360796069947…	128.119995	128.330002
6	[{'CENTROID_ID': 1, 'DISTANCE': 6.577599777882…	127.320000	129.350006
7	[{'CENTROID_ID': 1, 'DISTANCE': 6.561020502352…	129.570007	131.259995
8	[{'CENTROID_ID': 1, 'DISTANCE': 6.451268955571…	124.279999	130.000000
9	[{'CENTROID_ID': 1, 'DISTANCE': 6.558229665769…	128.399994	129.880005

	Low	Close	Volume
0	123.099998	123.739998	376189100
1	123.879997	128.300003	301726100
2	117.470001	122.860001	318086700
3	121.180000	124.580002	302589900
4	125.220001	129.369995	323230300
5	122.639999	125.175003	241795982
6	126.660004	128.500000	257883600
7	124.370003	126.459999	331964700
8	123.419998	130.000000	318333600
9	125.889999	127.250000	300087400

--- Cluster Centroid Analysis ---

	centroid_id	feature	numerical_value	categorical_value
0	1	std_Open	3.742058	[]
1	1	std_High	3.739191	[]
2	1	std_Low	3.747188	[]
3	1	std_Close	3.743012	[]
4	1	std_Volume	-0.343457	[]
5	2	std_Open	-0.012763	[]
6	2	std_High	-0.012892	[]
7	2	std_Low	-0.012268	[]
8	2	std_Close	-0.012620	[]
9	2	std_Volume	-0.626011	[]
10	3	std_Open	-0.335114	[]
11	3	std_High	-0.334504	[]
12	3	std_Low	-0.336635	[]
13	3	std_Close	-0.335518	[]
14	3	std_Volume	2.237179	[]
15	4	std_Open	-0.331353	[]
16	4	std_High	-0.331045	[]
17	4	std_Low	-0.332059	[]
18	4	std_Close	-0.331506	[]
19	4	std_Volume	0.143643	[]